

Les Threads

Présentation

« Un thread est une unité d'exécution faisant partie d'un programme. Cette unité fonctionne de façon autonome et parallèlement à d'autres threads. Le principal avantage des threads est de pouvoir répartir différents traitements d'un même programme en plusieurs unités distinctes pour permettre leurs exécutions "simultanée". » J.M. Doudoux

Pour créer un thread, votre classe peut hériter de la classe Thread.

Une méthode `run()` est de fait implémentée dans la classe. Elle contiendra l'ensemble des actions à exécuter par le thread.

La méthode `start()` d'un thread permet de démarrer le traitement défini dans `run()`.

Il est possible d'attendre la fin de l'exécution d'un thread en appelant la méthode `join()`.

Pour mettre un thread en sommeil il est possible d'utiliser la méthode static `sleep()` de la classe Thread.

Lorsque une ressource peut être utilisée par deux threads concurrents il est possible d'utiliser le mot clé `synchronized` pour obtenir un verrou et ainsi sécuriser son usage.

Pour aller plus loin...

- <https://docs.oracle.com/javase/10/docs/api/java/lang/Thread.html>
- <https://www.jmdoudoux.fr/java/dej/chap-threads.htm>
- <https://koor.fr/Java/Tutorial/Threads.wp>

Mise en pratique

Nous allons détailler l'exemple d'un jeu de lancé de dés où l'objectif est d'être le premier à atteindre le score de 50. Lorsqu'un joueur fait 6, le score de son adversaire est divisé par 2. Les joueurs lancent les dés toutes les demi-seconde. Voici la classe Player qui définit un joueur :

```
1  import java.util.Random;
2
3  public class Player extends Thread {
4      private String name;
5      private int score;
6      private Player adversaire;
7
8      public Player(String name, Player adversaire) {
9          this.name = name;
10         this.adversaire = adversaire;
11         this.score = 0;
12     }
13
14     private synchronized void play() {
15         Random random = new Random();
16         int tirage = random.nextInt(6) + 1;
17         this.score += tirage;
18         if (tirage == 6)
19             this.adversaire.penalite();
20         System.out.println(this);
21     }
22
23     public synchronized void penalite() {
24         if (this.score < 50) {
25             this.score = (int) this.score / 2;
26             System.out.println("Pénalité pour " + this);
27         }
28     }
29
30     public void run() {
31         while (this.score < 50) {
32             play();
33             try {
34                 sleep(500);
35             } catch (InterruptedException e) {
36                 e.printStackTrace();
37             }
38         }
39         System.out.println(this.name + " a terminé !!!");
40     }
41
42     public String toString() {
43         return this.name + " (score=" + this.score + ")";
44     }
45
46     public void setAdversaire(Player adversaire) {
47         this.adversaire = adversaire;
48     }
49
50 }
51 }
```

Afin de pouvoir faire jouer plusieurs joueurs en même temps la classe Player hérite de la classe Thread (l.3). La fonction run() (l.30 à 41) sera donc exécutée lorsque le thread Player sera démarré à l'aide de la fonction start() dans le main. Cette fonction va, tant que le score du joueur est inférieur à 50, jouer à l'aide de la fonction play() et attendre une demi-seconde. Comme les fonctions play() et penalite() peuvent vouloir accéder à la même ressource au même moment le mot clé synchronized est ajouté.

Pour jouer, la classe TestPlayer est la suivante :

```
1 public class TestPlayer {
2     public static void main(String[] args) {
3         Player p1 = new Player("Joueur 1", null);
4         Player p2 = new Player("Joueur 2", p1);
5         p1.setAdversaire(p2);
6
7         p1.start();
8         p2.start();
9
10        System.out.println("Début de la partie");
11        try {
12            p1.join();
13            p2.join();
14            System.out.println("Fin de la partie");
15        } catch (InterruptedException e) {
16            e.printStackTrace();
17        }
18    }
19 }
20 }
```

Après avoir instancié deux joueurs (l.3 à 5), on commence le jeu en démarrant le thread de chaque joueur à l'aide de la fonction `start()`. Après avoir démarré le jeu, le message "Début de la partie" sera affiché dans la console (l.10). Lorsque les deux threads seront terminés (fonction `join()`) le message "Fin de la partie" sera affiché (l.14).